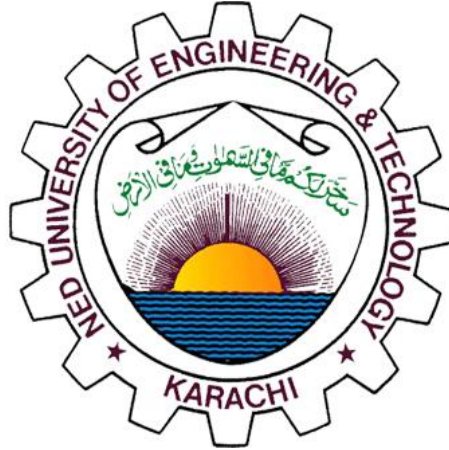


CCP-PROJECT



PROJECT NAME:

PARKING MANAGEMENT SYSTEM

GROUP MEMBERS:

RAMLA KAMRAN

CT-25051

RUSHDA ABBASI

CT-25074

COURSE NAME:

OBJECT ORIENTED PROGRAMMING

COURSE INSTRUCTOR:

DR. MURK MARVI

Parking Management System – C++ Project Report

Introduction

This project is a console-based Parking Management System developed in C++. The system demonstrates the implementation of important Object-Oriented Programming (OOP) concepts and Design Patterns.

The application manages vehicle parking operations such as:

- Parking vehicles
- Generating parking tickets
- Releasing parking slots
- Calculating parking fees
- Displaying available parking slots

The project uses multiple software design patterns including:

- Factory Pattern
- Singleton Pattern
- State Pattern
- Command Pattern

The main objective of this project is to simulate a real-world parking system while applying advanced OOP concepts in C++.

Objectives

The objectives of this project are:

1. To understand Object-Oriented Programming concepts.
 2. To implement design patterns in C++.
 3. To simulate real-life parking management operations.
 4. To practice dynamic memory allocation and polymorphism.
 5. To improve modular programming and code organization.
-

Features of the System

The system provides the following functionalities:

- Vehicle parking
 - Parking slot allocation
 - Ticket generation
 - Vehicle exit handling
 - Parking fee calculation
 - Slot availability checking
 - Separate slots for cars and bikes
-

Object-Oriented Concepts Used

1. Abstraction

Abstract classes are used to define common interfaces.

Example:

```
class Vehicle {  
public:  
    virtual void display() = 0;  
    virtual string getType() = 0;  
};
```

The `Vehicle` class hides implementation details and provides a common structure for all vehicle types.

2. Inheritance

Derived classes inherit properties and behaviors from base classes.

Example:

```
class Car : public Vehicle  
class Bike : public Vehicle
```

`Car` and `Bike` inherit from the `Vehicle` class.

3. Polymorphism

Runtime polymorphism is implemented using virtual functions.

Example:

```
Vehicle* v = VehicleFactory::createVehicle(type);
```

The program decides at runtime whether the object is a `Car` or a `Bike`.

4. Encapsulation

Classes combine data and functions together.

Example:

- `ParkingSlot`
- `ParkingTicket`
- `ParkingManager`

Each class manages its own responsibilities.

5. Dynamic Memory Allocation

Objects are created dynamically using the `new` keyword.

Example:

```
return new Car();
```

This allows runtime object creation.

Design Patterns Used

1. Factory Pattern

Purpose

The Factory Pattern is used to create objects without exposing object creation logic to the user.

Implementation

```
class VehicleFactory {  
public:  
    static Vehicle* createVehicle(string type)
```

Working

- If the user enters `car`, a `Car` object is created.
- If the user enters `bike`, a `Bike` object is created.

Benefits

- Reduces object creation complexity
 - Improves flexibility
 - Makes code easier to maintain
-

2. Singleton Pattern

Purpose

The Singleton Pattern ensures that only one object of `ParkingManager` exists.

Implementation

```
static ParkingManager* object;
```

Working

The method:

```
ParkingManager::getInstance()
```

returns the same object every time.

Benefits

- Centralized management
 - Prevents multiple managers
 - Saves memory
-

3. State Pattern

Purpose

The State Pattern changes slot behavior depending on whether it is occupied or available.

Classes Used

AvailableState
OccupiedState

Working

- Available slots return `Available`
- Occupied slots return `Occupied`

Benefits

- Cleaner state handling
 - Easy to extend
 - Better organization
-

4. Command Pattern

Purpose

The Command Pattern encapsulates user actions into separate command classes.

Classes Used

ParkCommand
ExitCommand
ShowSlotsCommand

Working

Each command contains its own `execute()` function.

Benefits

- Better modularity
- Easier command management
- Simplifies menu handling

Class Descriptions

1. Vehicle Class

Purpose

Acts as the abstract base class for all vehicles.

Functions

Function	Purpose
display()	Displays vehicle information
getType()	Returns vehicle type

2. Car Class

Purpose

Represents a car.

Functionality

- Displays car message
 - Returns vehicle type as `car`
-

3. Bike Class

Purpose

Represents a bike.

Functionality

- Displays bike message
 - Returns vehicle type as `bike`
-

4. VehicleFactory Class

Purpose

Creates vehicle objects dynamically.

Main Function

```
createVehicle(string type)
```

5. SlotState Class

Purpose

Abstract class for parking slot states.

Derived Classes

- AvailableState
 - OccupiedState
-

6. ParkingSlot Class

Purpose

Represents a parking slot.

Data Members

Variable	Purpose
slotID	Stores slot number
vehicle	Stores parked vehicle
state	Stores current slot state

Functions

Function	Purpose
isAvailable()	Checks slot availability
parkVehicle()	Parks a vehicle

Function	Purpose
removeVehicle()	Removes vehicle

7. ParkingTicket Class

Purpose

Represents a parking ticket.

Data Members

Variable	Purpose
ticketID	Unique ticket number
slotID	Assigned slot number
vehicle	Associated vehicle

Functions

Function	Purpose
printTicket()	Displays ticket details
calculateFee()	Calculates parking fee

8. ParkingManager Class

Purpose

Manages the entire parking system.

Responsibilities

- Managing slots
- Creating tickets
- Assigning slots
- Releasing slots
- Showing available slots

Important Functions

Function	Purpose
getInstance()	Returns singleton object

Function	Purpose
assignSlot()	Assigns parking slot
createTicket()	Generates ticket
releaseSlot()	Frees parking slot
showAvailableSlots()	Displays free slots

9. Command Classes

Purpose

Handle menu operations.

Classes

Class	Responsibility
ParkCommand	Parks vehicle
ExitCommand	Handles vehicle exit
ShowSlotsCommand	Displays free slots

Relationships Between Classes

Association

Example

```
ParkingTicket ---> Vehicle
```

A ticket is associated with a vehicle.

Aggregation

Example

```
ParkingManager ---> ParkingSlot
```

The manager stores parking slots.

Inheritance

Example

```
Car ---> Vehicle  
Bike ---> Vehicle
```

Dependency

Example

```
ParkCommand ---> VehicleFactory
```

The command depends on the factory to create vehicles.

Working of the System

Step 1 – User Chooses Option

The menu is displayed:

```
1. Park  
2. Exit  
3. Show Available Slots  
0. Quit
```

Step 2 – Parking Vehicle

- User enters vehicle type.
- Factory creates vehicle object.
- ParkingManager assigns slot.
- Ticket is generated.

Example:

```
Enter vehicle type: car  
car parked at slot: 1  
Ticket ID: 1, Slot: 1
```

Step 3 – Vehicle Exit

- User enters ticket ID.
- System calculates fee.
- Slot becomes available again.

Example:

```
Enter Ticket ID: 1
Enter hours: 3
Parking Fee: Rs 60/-
Vehicle exited.
```

Step 4 – Show Available Slots

Displays all free slots.

Example:

```
Bike Slots: 1 2 3
Car Slots: 2 3 4 5
```

Advantages of the Project

- Demonstrates real-world OOP implementation
 - Uses multiple design patterns together
 - Modular and organized structure
 - Easy to extend with new vehicle types
 - Better code reusability
 - Good example of polymorphism and abstraction
-

Limitations

- Memory is not released properly using destructors
 - No file handling
 - No database integration
 - No graphical user interface
 - Ticket deletion is not implemented
-

Possible Future Improvements

The project can be improved by adding:

1. File handling for saving records
 2. Database connectivity
 3. GUI interface
 4. Multiple parking floors
 5. Online payment system
 6. Security authentication
 7. Destructor implementation
 8. Smart slot reservation
 9. Time and date tracking
 10. Vehicle number registration
-

Conclusion

The Parking Management System successfully demonstrates the implementation of advanced Object-Oriented Programming concepts and Design Patterns in C++.

The project provides a practical simulation of a real-world parking environment. Through this system, concepts such as abstraction, inheritance, polymorphism, encapsulation, dynamic memory allocation, and software design patterns are applied effectively.

This project is useful for understanding how OOP and design patterns work together to create scalable and maintainable software systems.

Technologies Used

Technology	Purpose
C++	Programming Language
OOP	Software Design
STL Vector	Dynamic Storage
Dynamic Memory	Runtime Object Creation

Sample Output

```
1. Park
2. Exit
3. Show Available Slots
0. Quit
Choice: 1

Enter vehicle type: car
Car entered the parking.
car parked at slot: 1
Ticket ID: 1, Slot: 1

Choice: 3
Bike Slots: 1 2 3 4 5
Car Slots: 2 3 4 5

Choice: 2
Enter Ticket ID: 1
Enter hours: 2
Parking Fee: Rs 40/-
Vehicle exited.
```

References

1. Object-Oriented Programming in C++
2. Design Patterns by Gang of Four (GoF)
3. C++ Standard Template Library Documentation
4. Software Engineering Design Principles